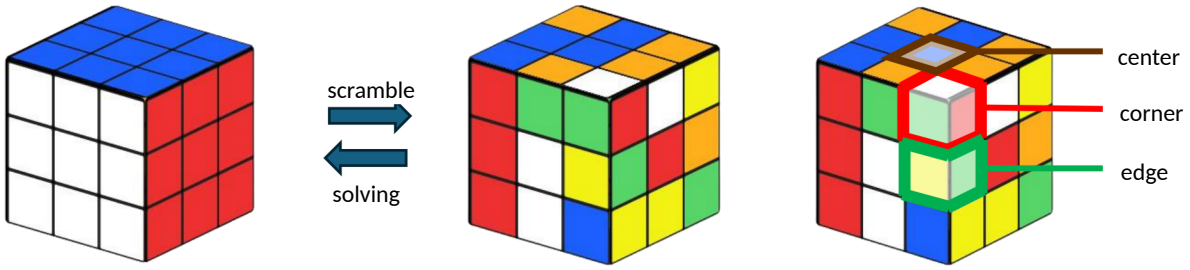
Searching for optimal solutions to the Rubik's Cube

The Rubik's Cube was invented in 1974 by Ernö Rubik, and has since become an iconic puzzle, notably for its great complexity. Many algorithms have been devised to solve it, some of which seek to solve the Rubik's Cube optimally, that is, in a minimal number of moves. This approach makes the problem particularly hard (it has been shown more generally that the associated decision problem for an $n \times n \times n$ Rubik's Cube is NP-complete [1]), and optimal solving algorithms are limited in their ability to solve some positions that are particularly far from the solved position. Other algorithms therefore restrict themselves to searching for near-optimal solutions, that is, solutions that use a number of moves close to the minimum required.

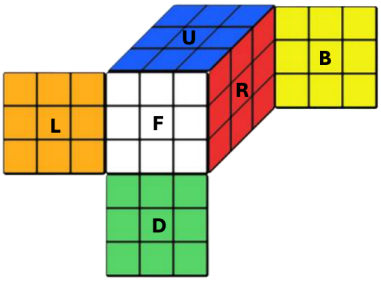
In this TIPE, we aim to compare two solving algorithms, one optimal and the other near-optimal, in terms of the length of the solutions they find, as well as their time and space complexity.

I- The Rubik's Cube and its group

Let us first introduce the Rubik's Cube puzzle in more detail. The Rubik's Cube is a cube whose six faces are colored with six distinct colors in the solved state. It is made of 27 smaller cubes, or cubies (one of which is at the center of the cube), which, thanks to the independent movement of the faces, can be scrambled. Solving the Rubik's Cube therefore means bringing it back from any position, where the cubies are scrambled, to the solved state, where each face has a single color. We can immediately distinguish three types of cubies, according to their number of stickers: the 6 "centers" (one per face) have one sticker, the 12 edges have two, and finally the 8 corners have three. We can consider, and this is what we will do from now on, that the centers are fixed and that only the corners and edges move.



To make the Rubik's Cube easier to describe, we assign a letter to each face: F, U, L, R, D, B, for Front, Up, Left, Right, Back and Down. Corners and edges are then denoted by the letters of the faces around them. We then define 18 moves on the Rubik's Cube, which correspond to rotations of one, two or three quarter turns of each of the six faces. They are written F, F2, F3, B, B2, etc., where the letter denotes the face being turned and the number the number of quarter turns, performed clockwise. Whenever we later speak of a distance in number of moves, these are, unless stated otherwise, the moves we allow ourselves.



The Rubik's Cube can also be seen as a group, which turns out to be central in the design of computer solving algorithms. Let us first define

$$E \coloneqq \{ \textit{ set of all positions of the Rubik's Cube } \}$$

E is the set of *all* positions of the Rubik's Cube, that is, not only those reachable by legal moves from the solved position, but also those obtained by taking the Rubik's Cube apart and putting it back together. According to our previous description of the Rubik's Cube, only the corners and the edges move, and they cannot be swapped with each other since they do not have the same number of stickers. We can therefore see each position of the Rubik's Cube as a pair made of a position of the corners and a position of the edges. Thus $E = E_C \times E_A$,

$$\text{where } \begin{cases} E_C \coloneqq \{\textit{set of positions of the corners of the Rubik's Cube}\} \\ E_A \coloneqq \{\textit{set of positions of the edges of the Rubik's Cube}\} \end{cases} .$$

We can then define the group of the Rubik's Cube $(G, \circ)$. The elements of G are the scrambles, that is, the maps from E to E that move corners and edges, and $\circ$ is the composition law. Similarly, we can define the groups of corner and edge scrambles G_C and G_A, so that every scramble in G decomposes into two maps, the first acting on E_C and moving the corners, the second acting on E_A and moving the edges.

We can describe G even more precisely. Take the corners, for example: we notice that every scramble of the corners can be decomposed into two steps: first, we can permute the corners, then rotate each of them in place to give it a certain orientation. One is even easily convinced that this decomposition is unique. This is what the following mathematical statement expresses:

$$\forall g \in G,\ \exists!\ (\sigma, \rho) \in \textit{Perm}_C \times \textit{Rot}_C,\ \ g = \rho \circ \sigma,$$
$$\text{where } \begin{cases} \textit{Perm}_C \coloneqq \{g \in G \mid g \textit{ does not change the orientation of the corners}\} \\ \textit{Rot}_C \coloneqq \{g \in G \mid g \textit{ does not permute the corners}\} \end{cases} .$$

This amounts to saying that $G_C = \textit{Perm}_C \rtimes \textit{Rot}_C$. The elements of $\textit{Perm}_C$ correspond to the permutations of the 8 corners, so $\textit{Perm}_C \cong \mathfrak{S}_8$. The elements of $\textit{Rot}_C$ can be described by the number of rotations by which each of the corners, which have three possible orientations, must be turned, so $\textit{Rot}_C \cong \left(\frac{\mathbb{Z}}{3\mathbb{Z}}\right)^8$. We can proceed in the same way for G_A, which allows us to prove that $G_A = \textit{Perm}_A \rtimes \textit{Rot}_A$ with $\textit{Perm}_A \cong \mathfrak{S}_{12}$ and $\textit{Rot}_A \cong \left(\frac{\mathbb{Z}}{2\mathbb{Z}}\right)^{12}$.

One may wonder, at first, what this complicated detour through groups is good for. First of all, note that we can identify the sets of scrambles and of positions, G and E. To do so, it suffices to consider the bijection

$$\begin{aligned} \psi : G &\to E \\ g &\mapsto g(I) \end{aligned}$$

Thanks to our decomposition of G into 4 subgroups, and since we identify scrambles with positions, this means that any position can be represented by 4 pieces of data: the orientations and permutations of the corners, and the orientations and permutations of the edges. This will be central in our practical implementation of the Rubik's Cube. Moreover, the group approach to the Rubik's Cube leads to the so-called fundamental theorem, which gives a necessary and sufficient condition for a position to be legal, that is, reachable from the solved state by legal moves:

$$\text{A position is legal} \iff \begin{cases} \sum_{i=1}^{8} o_{c_i} = 0 \\ \sum_{i=1}^{12} o_{a_i} = 0 \\ \epsilon(\sigma_c) = \epsilon(\sigma_a) \end{cases}$$

where $(o_{c_i})_{1 \leq i \leq 8}$, $(o_{a_i})_{1 \leq i \leq 12}$ denote the orientations of the corners and edges relative to the solved position, and σ_c and σ_a the permutations of the corners and of the edges.

This allows us to estimate the complexity of this puzzle, which therefore has on the order of

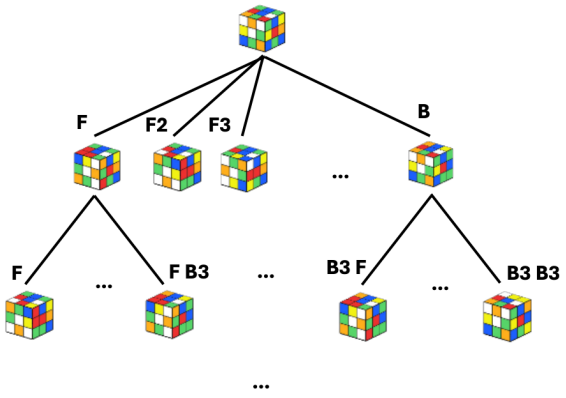
$$3^7 \times 2^{11} \times \frac{8! \times 12!}{2} \approx 4.3 \times 10^{19} \text{ positions.}$$

Finally, it is notably by using this group approach to the Rubik's Cube that a team of researchers managed to prove in 2010 that every legal position can be solved in 20 moves or fewer [2].

II- Optimal solving algorithms

Let us now return to our initial problem: that of an algorithm able to solve any position of the Rubik's Cube in a minimal number of moves.

One may first be tempted to take a brute-force approach: to obtain an optimal solution, it is indeed enough to go through all the sequences of moves starting from the position to solve, by increasing length, until the solved position is reached. This can be visualized as a tree whose root is the position to solve and in which every node has 18 children, which are the 18 positions obtained from that node by applying the 18 moves. Of course, several distinct nodes can correspond to the same position in such a tree.

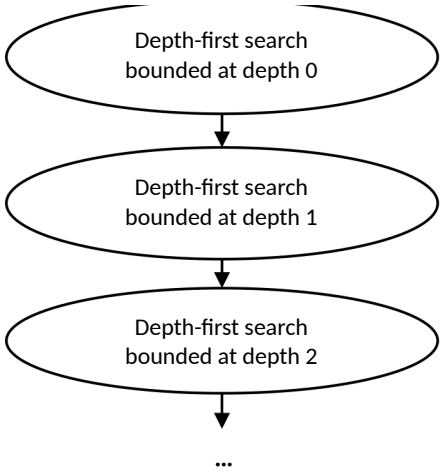


Finding an optimal solution to the Rubik's Cube is then simply a matter of finding a shortest path between the root and one of the solved positions appearing among its descendants. A breadth-first search, which explores the nodes by increasing depth, is therefore enough to find an optimal solution.

However, the high arity of the nodes (which is 18) will limit us in practice: on the one hand, the number of nodes, and therefore the time complexity, grows exponentially with the depth of the position to solve; on the other hand, the number of nodes to store in the queue required by the breadth-first search, and therefore the memory used by the algorithm, also grows exponentially. Experimentally, we are limited to depth 6 or 7.

Korf's algorithm [3] relies on this same tree, in which we must find the shortest path between the root and one of the solved positions appearing among its descendants, but it uses the IDA* (Iterative Deepening A*) algorithm as its search algorithm, which solves the space and time complexity problems encountered above.

First, the Iterative Deepening aspect solves the space complexity problem. As the name suggests, we use depth-first searches, which are much less costly in space than breadth-first search. We are however faced with a problem: since depth-first search does not visit the nodes by increasing depth, how can we guarantee that the solution found is optimal? This is where the iterative aspect comes into play: we perform a sequence of depth-first searches bounded at a certain exploration depth, gradually increasing this bound until a solution is found.



We therefore explore all the nodes at depth 0, then all the nodes at depths 0 and 1, then all the nodes at depths 0, 1 and 2, etc.: in this way, the exploration always proceeds by increasing depth. We thus guarantee that the solution found is optimal, while limiting the number of nodes to be explored in the stack, which is 341 in the worst case.

However, we still explore all the nodes of a level before moving on to the next level, and the time complexity remains exponential for now. This is where the A* aspect solves the problem. We build admissible heuristics, that is, functions that take a position and give an underestimate of the number of moves needed to solve it. So let h be such a heuristic, and suppose we explore a node p at depth i during the depth-first search bounded at depth j: if $h(p) + i > j$, then, since the heuristic h is admissible, this means that we will not find the solved position among the descendants of this node during this depth-first search. We can therefore prune this branch.

To build such heuristics, Korf proposes to solve "subproblems" of the Rubik's Cube. For example, he proposes the following heuristic

$$h_2 := \max\big(\textit{no. of moves to solve the corners},$$
$$\textit{no. of moves to solve 6 of the edges},\ \textit{no. of moves to solve the 6 other edges}\big)$$

It is therefore clear that this heuristic is admissible. Korf's approach then consists in solving all the configurations of these subproblems beforehand, and storing the values obtained for the heuristic in large databases [4]. It thus appears that the choice of heuristic results from a trade-off between space and time complexity: the more a heuristic lets us go fast, and therefore prune branches, the more information it gives and the larger the databases needed to store its values. Korf thus conjectured in 1997 that the execution time of his algorithm is inversely proportional to the memory used for its databases [3]. This is what I wanted to verify in this TIPE, and to do so I built 5 heuristics with an increasing number of entries in their databases.

Name of the heuristic	Number of entries in the heuristic's databases
h_1	89 634 816
h_2	173 335 780
h_3	652 337 280
h_4	1 110 049 920
h_5	1 590 533 496

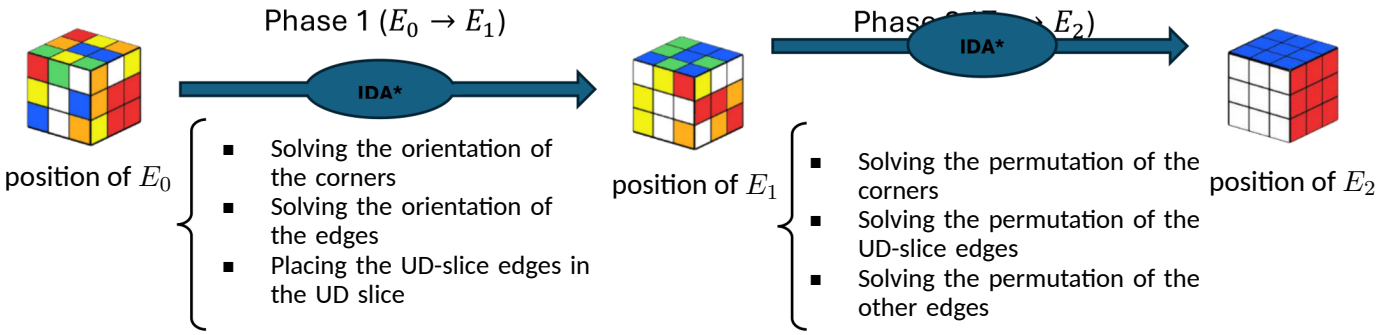
Finally, Kociemba's "two-phase" algorithm [5] proposes to relax the optimality constraint by searching for near-optimal solutions, that is, solutions that use a number of moves close to the minimum required. To do so, we go back to the group of the Rubik's Cube and define a decreasing sequence of subgroups with well-chosen properties. Let

- $G_0 := < U,\ D,\ R,\ L,\ F,\ B >$ and $E_0 := \psi(G_0)$
- $G_1 := < U,\ D,\ R2,\ L2,\ F2,\ B2 >$ and $E_1 := \psi(G_1)$
- $G_2 := \{id\}$ and $E_2 := \psi(G_2)$

E_0 is the set of positions reachable by legal moves from the solved position, so it is the set of legal positions. It is also clear that the set E_2 reduces to the solved state. Finally, one can prove that the positions of E_1 satisfy the following properties, which are central to Kociemba's algorithm:

- The corners and edges of the positions of E_1 are correctly oriented, that is, oriented as in the solved state
- The edges of the UD slice (between the U and D faces) are located in that slice

Kociemba's algorithm then consists in taking the position to solve, any legal position of E, and bringing it in a first phase to a position of E_1, then bringing this new position to a position of E_2, that is, solving the Rubik's Cube.



To avoid undoing the solving work done in the first phase, in the second phase we make sure to always stay within the subset E_1 by using only the moves of G_1. The phase transitions are carried out with the IDA* algorithm, exactly as in Korf's algorithm, with heuristics based on solving subproblems, computed beforehand and stored in databases. Since we force the search to go through the position of E_1 closest to the position to solve, we can no longer guarantee that the solution found is optimal; however, it has been proven that the maximum numbers of moves for the two phases are 12 and 18, so the solution obtained by this algorithm never exceeds 30 moves (and in practice this maximum is very rarely reached), which justifies the name "near-optimal" solving.

III- Putting the algorithms into practice

Before implementing solving algorithms, we first need a computer representation of the Rubik's Cube. For this, I went back to the group approach to the Rubik's Cube and defined the following `pos` structure to represent the Rubik's Cube in C, which contains 4 arrays for the permutations and orientations of the corners and edges.

```c
struct pos {
    int pc[8];
    int oc[8];
    int pa[12];
    int oa[12];
};
typedef struct pos pos;
```

More precisely, taking the corners as an example, we first had to assign a number and a reference face to each corner. Then, for an arbitrary scramble of the corners, here is what the corner permutation array `pc` (top) and corner orientation array `oc` (bottom) can look like.

Index	0	1	2	3	4	5	6	7
Value	5	6	7	4	0	1	3	2

Index	0	1	2	3	4	5	6	7
Value	0	0	1	1	2	0	2	0

At index i, `pc` gives the number of the corner that replaces corner number i in this scramble. Similarly, `oc`(i) is the orientation of the corner that replaces corner i in this scramble, the orientation being defined by the positions of the marked faces of the replaced corner and of the corner replacing it, as illustrated below.



Orientation: 0



Orientation: 1



Orientation: 2

In black: marked face of the replaced corner

In red: marked face of the corner replacing it

We can describe the 18 moves of the Rubik's Cube in the same way, with permutation and reorientation arrays. Since we identify scrambles with positions, this gives very simple formulas for applying a move to a position, which simply corresponds to the composition of two scrambles.

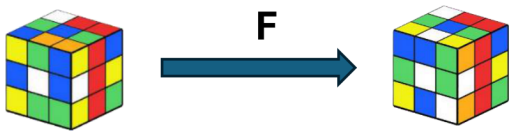
Finally, we define a second representation of the Rubik's Cube, which corresponds to the first one, but in a compressed form (we will explain later why we need it). To do so, we compress the information contained in the 4 arrays into 4 integers, using various combinatorial tricks.

```c
struct pos_comp {
    uint16_t pc;
    uint16_t oc;
    uint32_t pa;
    uint16_t oa;
};
typedef struct pos_comp pos_comp;
```

For example, the edge orientation array is an array of 0s and 1s, which can be seen as a binary number, which we then convert to decimal. For the permutations, we use a Lehmer code to turn them into tuples of $[\![1,1]\!] \times [\![1,2]\!] \times \cdots \times [\![1,n]\!]$, which can then be seen as numbers in the factorial number system, also converted to decimal.

Now that we have a working Rubik's Cube, we can move on to the practical implementation of the algorithms. For each of the two algorithms, the first step is to generate the heuristics used to run IDA*, whose values are stored in databases. For these databases, I chose simple text files, with one number per line, corresponding to the value of the heuristic for a given position. This therefore requires building a perfect hash function for each of the databases, which gives the index of the line to read in the text file for the position given as input. Using intricate combinatorial methods, we manage to build such hash functions, and to make them minimal as well, that is, they use no more lines than necessary.

We can then generate the values of the heuristic. I first got stuck on this point, because I initially thought that, since the subproblems are much less complex, they could be solved one by one by brute force: when putting it into practice, I finally estimated the time required at a few millennia! What was actually needed was dynamic programming: we initialize the value of the heuristic to the maximum it can take for all positions, except for the solved position, for which we set it to 0, then we create a queue that initially contains only the solved position, and which will let us update the values of the heuristic step by step.



Position p_1 Position p_2

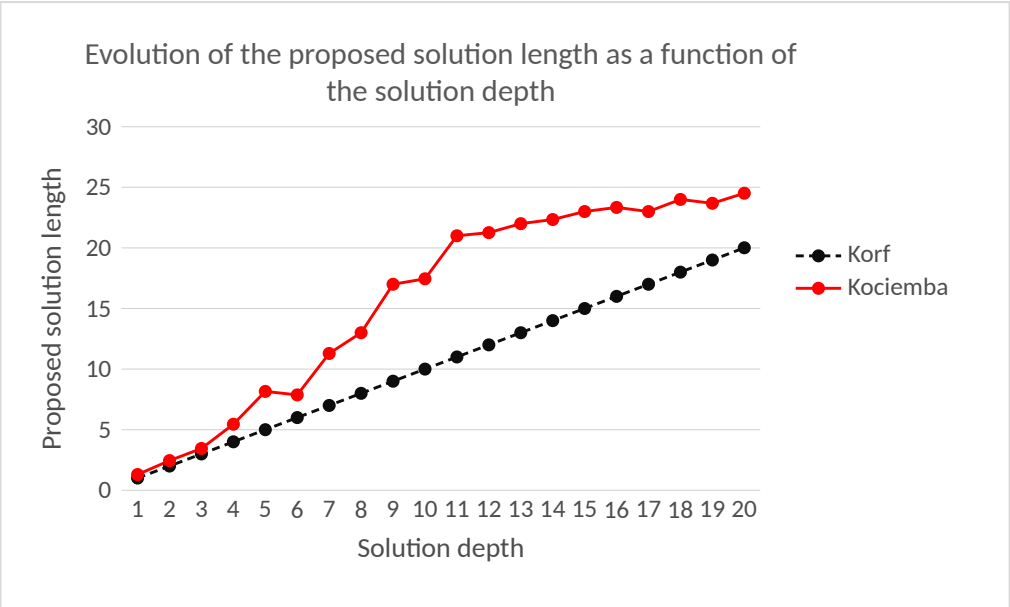
For example, if in this situation *heur*$(p_2) > $ *heur*$(p_1) + 1$, we can update the value of *heur*(p_2) to *heur*$(p_1) + 1$, then add p_2 to the queue to see whether this update changes the computed value of the heuristic for the neighbors of p_2.

Once the databases have been generated, writing Korf's and Kociemba's programs is straightforward.

IV- Results

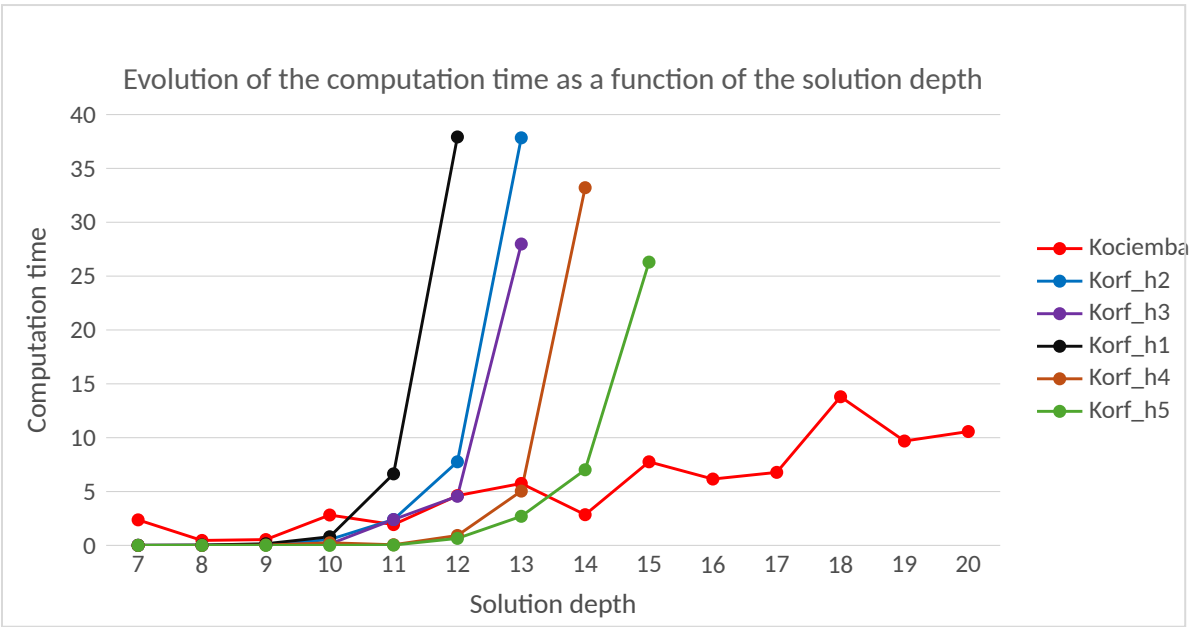
In this part, we compare the performance of our two implementations of Korf's and Kociemba's algorithms.

We can first compare these algorithms in terms of the length of the solution they find. The following data were generated by running the algorithms on a dataset of positions at different depths.



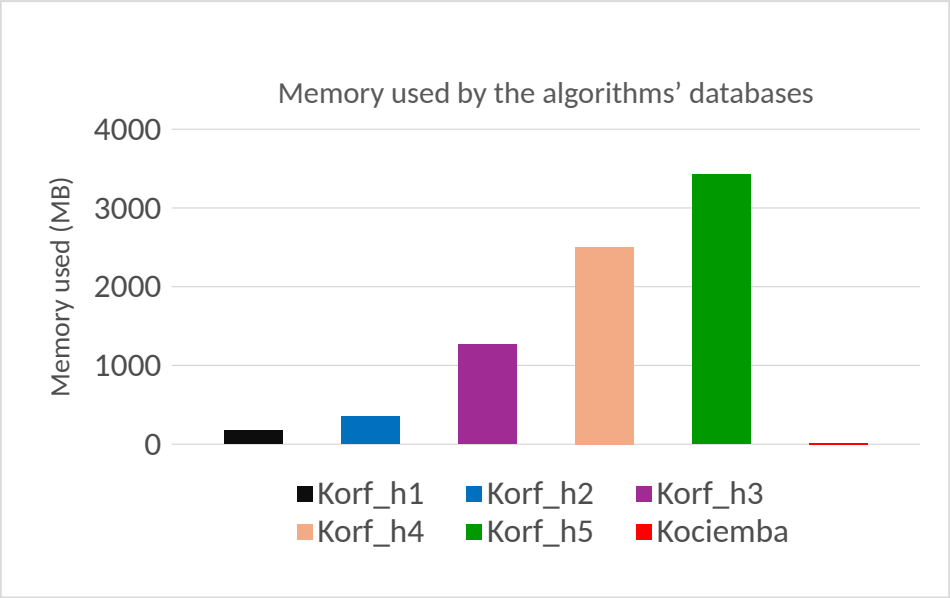
Since Korf's algorithm is optimal, it always returns a solution whose length equals the depth of the position to solve. Kociemba's algorithm, on the other hand, finds a near-optimal solution, never exceeding 25 moves on our dataset. The difference between the length of the solution found by Kociemba's algorithm and the minimum required is particularly large for positions of "intermediate" depth, roughly between 8 and 14, and becomes less and less significant for positions further away from the solved position.

Let us now compare these algorithms in terms of time complexity. Since they use complicated heuristics, it cannot be computed in the general case, so we analyze the execution times as a function of depth.



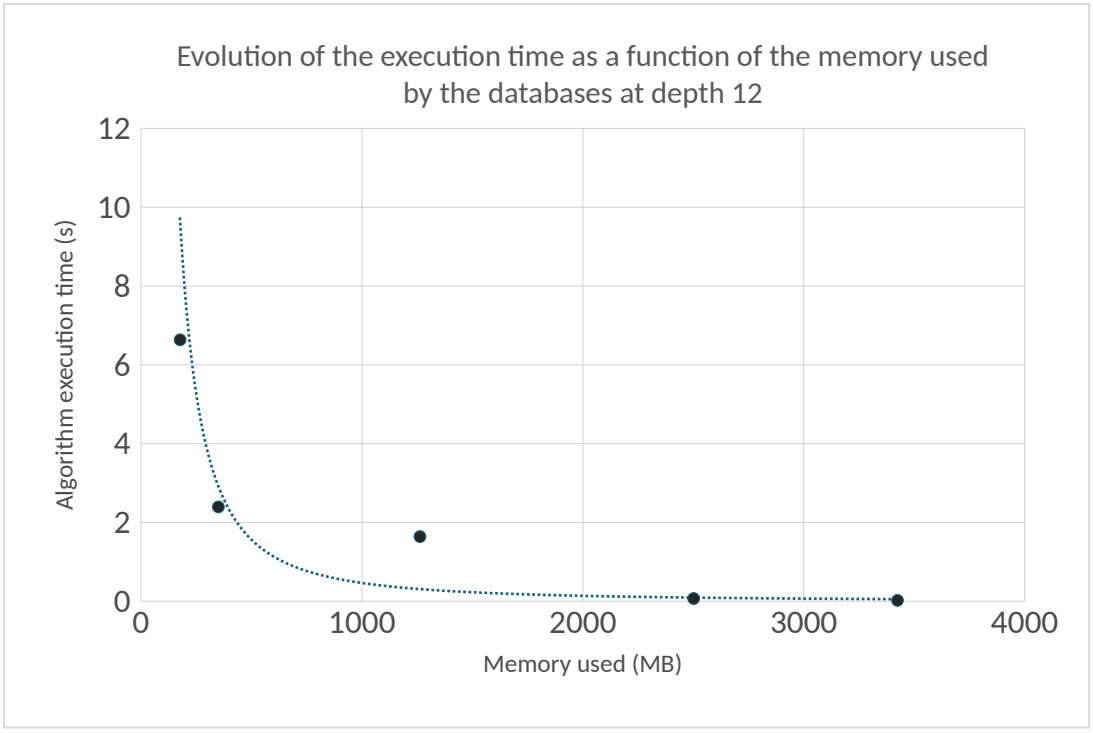
Only Kociemba's algorithm manages to solve any position of the Rubik's Cube, and even though its execution time grows with depth, it stays under 15 seconds even for the deepest solutions, which is very reasonable. Korf's algorithm, on the other hand, is limited: even with the best heuristic, it cannot keep a reasonable time from depth 15 onwards. It works well on shallow and even intermediate positions,

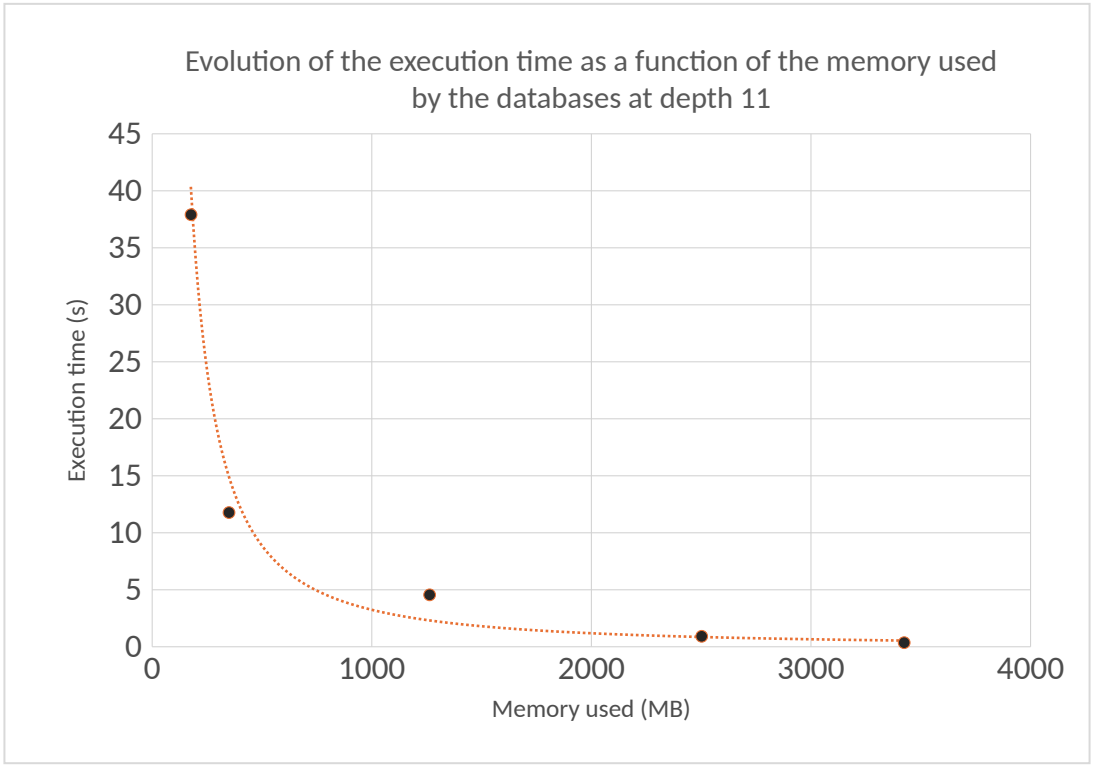
but beyond that, its execution time seems to grow exponentially. We can clearly see the influence of the amount of memory used by the databases on the execution time of the algorithms, going from a maximum solving depth of 12 for the algorithm using the smallest databases to 15 for the one with the largest databases. This leads us to consider the space complexity of each of these algorithms.



As intended, the 5 heuristics of Korf's algorithm correspond to an increasing amount of memory used. What stands out above all is that the amount of memory used by Kociemba's algorithm is tiny compared with Korf's algorithms, even when considering Korf's simplest heuristic.

This brings us to Korf's conjecture, which states that the execution time of Korf's algorithm is inversely proportional to the memory used by the databases. I decided to try to verify it experimentally at two intermediate depths where all the algorithms can solve the Rubik's Cube in a reasonable time. We do obtain a hyperbolic profile, but one that is better approximated by n^{α} with α close to 1.5. In any case, the idea behind Korf's conjecture, namely that using more memory first reduces the execution time quite quickly, and then becomes less and less worthwhile, is confirmed.





V- Conclusion

Thus, optimal solving algorithms such as Korf's, despite their theoretical interest, are limited in practice because they can only be applied to positions that are not too far from the solved position. Their speed can however be improved by sacrificing memory, with a gain that is significant at first and then less and less worthwhile. Kociemba's algorithm, which relaxes the optimality constraint, provides solutions in an always reasonable time and with minimal memory consumption: the fact that it is the most widely used computer algorithm for solving the Rubik's Cube comes as no surprise.

Bibliography

[1] Erik D. Demaine, Sarah Eisenstat, and Mikhail Rudoy. Solving the Rubik's Cube Optimally is NP-complete. In 35th Symposium on Theoretical Aspects of Computer Science (STACS 2018). Leibniz International Proceedings in Informatics (LIPIcs), Volume 96, pp. 24:1-24:13, Schloss Dagstuhl - Leibniz-Zentrum für Informatik (2018)

[2] Tomas Rockiki, Herbert Kociemba, Morley Davidson, John Dethridge, The diameter of the Rubik's Cube group is twenty, SIAM J. Discrete Math. 27 (2) (2013) 1082-1105.

[3] Richard E. Korf. Finding optimal solutions to Rubiks cube using pattern databases. In Proc. 19th National Conference on Artificial Intelligence AAAI, 1997, pp. 700-705.

[4] Joseph Culberson, Jonathan Schaeffer, Pattern databases, Comput. Intell. 14 (3) (1998) 318-334.

[5] Herbert Kociemba. The two-phase-algorithm. `http://kociemba.org/cube.htm`